Lab Report Module 10 - Principal Component Analysis

EE-512 - Applied Biomedical Signal Processing - Fall 2025

January 19, 2026

Ahmed Boubakry, Johann Clausen, Alice Gonzalez, Nolan Röder, Margot Romelli, Giovanni Tamò

Instructions

- Please **submit** your **report** as a **single PDF** file.
- We recommend working in **groups of 3 students**; the last group can be a group of 2 or 4.
- You can **prepare one single report** for the group (*name1_name2_name3_lab_PCA.pdf*), but **every member needs to upload** the same file individually.
- There are **2 experiments** in this practical session. The **Python** code for each experiment is already coded and will be provided as **Jupyter notebooks**. These will only **require minimal input** from you. A major part of this practical session is thus focused on questions testing your understanding and correct interpretation of the signals and the analysis results that you see.

1 PCA for Source Separation of Abdominal ECG Signals

1.1 Introduction

In this exercise we use PCA for the separation of maternal and fetal electrocardiography (ECG) signals in abdominal ECG (aECG) data recorded on the belly of a pregnant woman. Due to the low signal strengh of fetal ECG (fECG) signals it is an "algorithmic challenge" to properly separate fECG from much stronger maternal ECG (mECG) signals [1].

The present example uses a simplified version of the method proposed by Varanini et al. [2].

1.2 References

[1] R. Kahankova et al., "A Review of Signal Processing Techniques for Non-Invasive Fetal Electrocardiography," IEEE Reviews in Biomedical Engineering, vol. 13, pp. 51–73, 2020, doi: 10.1109/RBME.2019.2938061.

[2] M. Varanini, G. Tartarisco, L. Billeci, A. Macerata, G. Pioggia, and R. Balocchi, "An efficient unsupervised fetal QRS complex detection from abdominal maternal ECG," Physiol. Meas., vol. 35, no. 8, pp. 1607–1619, Aug. 2014, doi: 10.1088/0967-3334/35/8/1607.

[3] Source of data: https://physionet.org/content/challenge-2013/1.0.0/

1.3 PCA Analysis Results

[1]:
```
import numpy as np
import pandas as pd
from scipy.signal import filtfilt, butter
from sklearn.decomposition import PCA

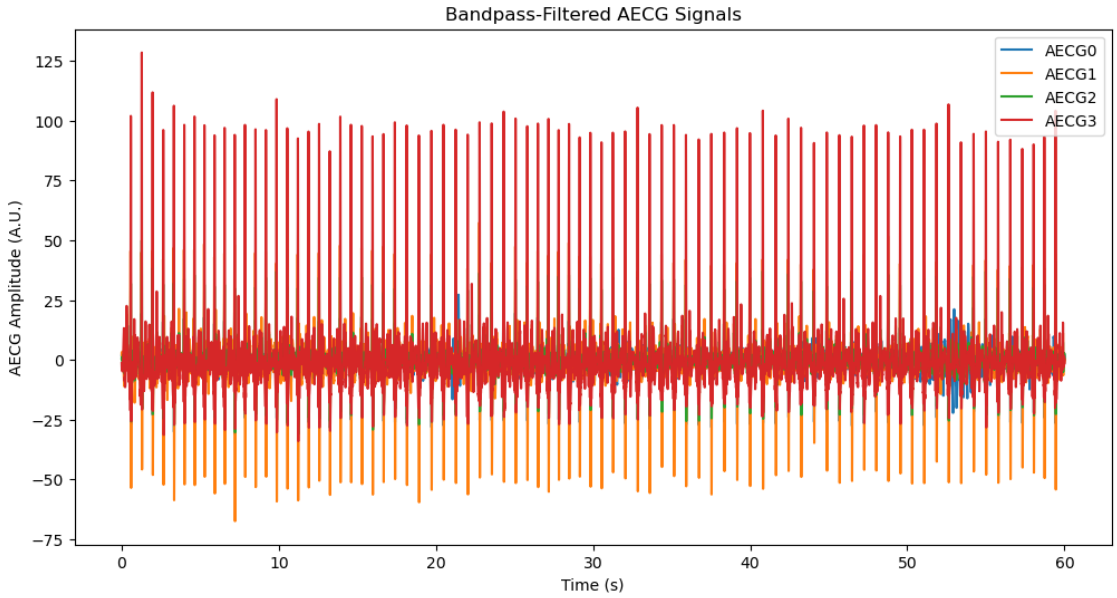
import matplotlib.pyplot as plt

from mqrs_utils import cancel_mqrs
from ecgdetectors import Detectors
```

[2]:
```
# load abdominal ECG (aECG) data
# transformed from initial source: https://physionet.org/content/challenge-2013/
  ↪1.0.0/set-a/a13.dat
filename = 'aecg_a13.hdf5'
aecg = pd.read_hdf(filename, key='signals').values
fs = 1000
t = np.arange(aecg.shape[0]) / fs
```

[3]:
```
# bandpass filter data
b, a = butter(4, np.asarray([3, 45]), fs=fs, btype='bandpass')
aecg = filtfilt(b, a, aecg, axis=0)

# plot
plt.figure(figsize=(12, 6))
for i in range(aecg.shape[1]):
    plt.plot(t, aecg[:, i], label=f'AECG{i}')
plt.xlabel('Time (s)')
plt.ylabel('AECG Amplitude (A.U.)')
plt.title('Bandpass-Filtered AECG Signals')
plt.legend()
plt.show()
```

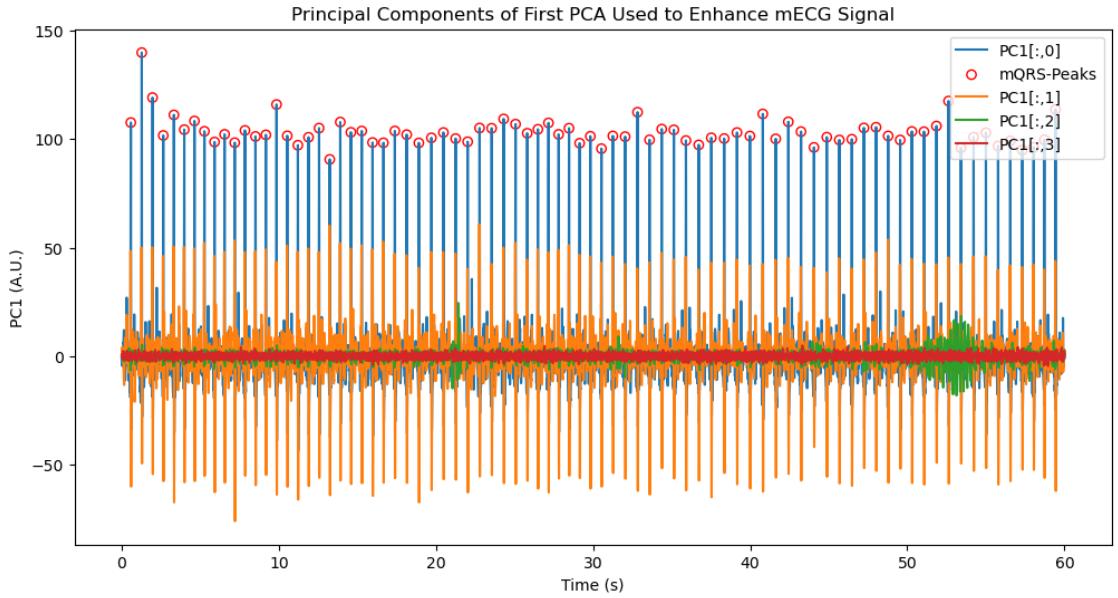


[4]:
```
# apply first PCA for enhancing maternal ECG component
pca1 = PCA()
pc1 = pca1.fit_transform(aecg)

# maternal ECG as the first principal component, note that this
# remains a guess and would need to be automated in the final solution
maternal_ecg = pc1[:, 0]

# detect maternal QRS peaks
mqrs_peaks = Detectors(fs).engzee_detector(maternal_ecg)

# plot
plt.figure(figsize=(12, 6))
for i in range(pc1.shape[1]):
    plt.plot(t, pc1[:, i], label=f'PC1[:,{i}]')
    if i == 0:
        plt.scatter(t[mqrs_peaks], pc1[mqrs_peaks, i], color='red', marker='o',␣
 ↪facecolors='none', label='mQRS-Peaks')
plt.xlabel('Time (s)')
plt.ylabel('PC1 (A.U.)')
plt.title('Principal Components of First PCA Used to Enhance mECG Signal')
plt.legend()
plt.show()
```



[5]:
```
# remove maternal QRS complexes from signal to obtain a best possible fetal ECG␣
 ↪signal
x_residual, mecg_estimations = cancel_mqrs(fs, pc1, np.asarray(mqrs_peaks))

# plot
fig, axs = plt.subplots(3, 1, figsize=(12, 10), sharex=True)

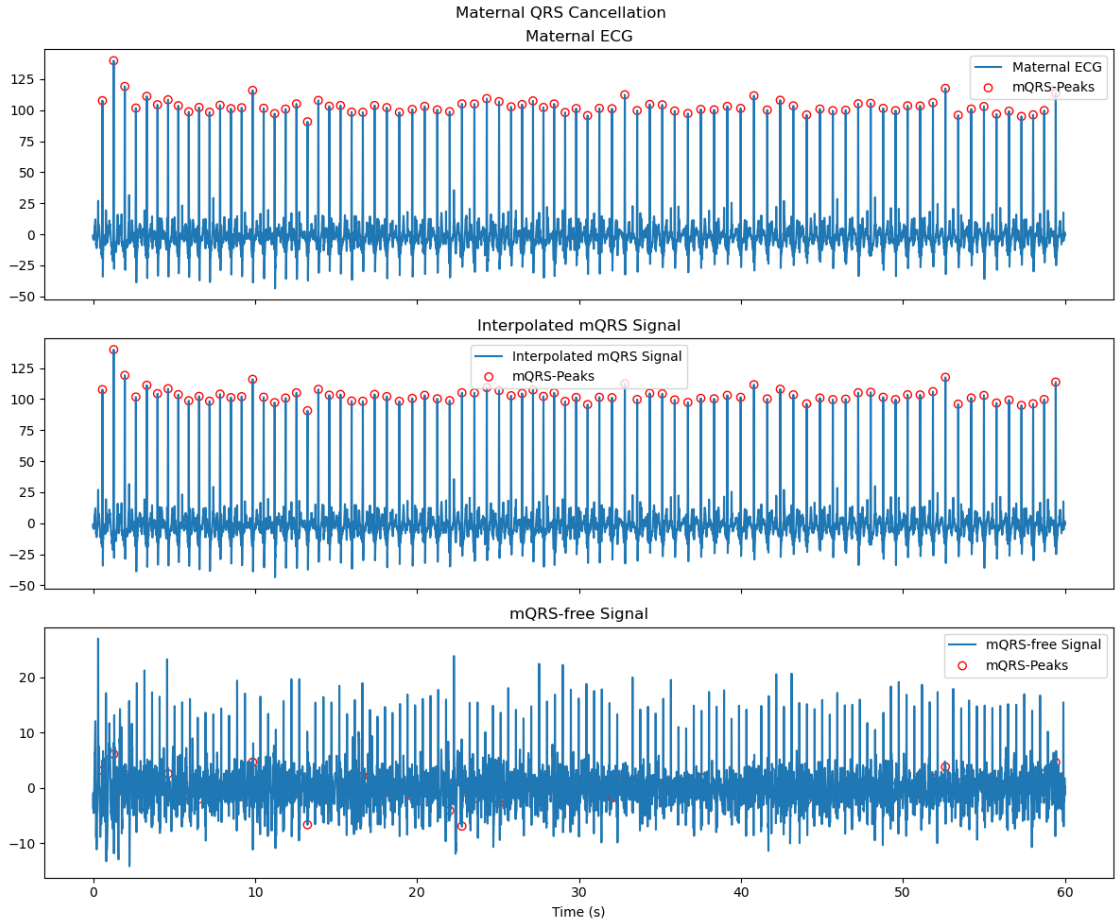
axs[0].plot(t, pc1[:,0], label='Maternal ECG')
axs[0].scatter(t[mqrs_peaks], pc1[mqrs_peaks, 0], color='red', marker='o',␣
 ↪facecolors='none', label='mQRS-Peaks')
axs[0].set_title('Maternal ECG')
axs[0].legend()

axs[1].plot(t, mecg_estimations[:, 0], label='Interpolated mQRS Signal')
axs[1].scatter(t[mqrs_peaks], mecg_estimations[mqrs_peaks, 0], color='red',␣
 ↪marker='o', facecolors='none', label='mQRS-Peaks')
axs[1].set_title('Interpolated mQRS Signal')
axs[1].legend()

axs[2].plot(t, x_residual[:, 0], label='mQRS-free Signal')
axs[2].scatter(t[mqrs_peaks], x_residual[mqrs_peaks, 0], color='red',␣
 ↪marker='o', facecolors='none', label='mQRS-Peaks')
axs[2].set_title('mQRS-free Signal')
axs[2].set_xlabel('Time (s)')
axs[2].legend()

plt.suptitle('Maternal QRS Cancellation')
```

```
plt.tight_layout()
plt.show()
```

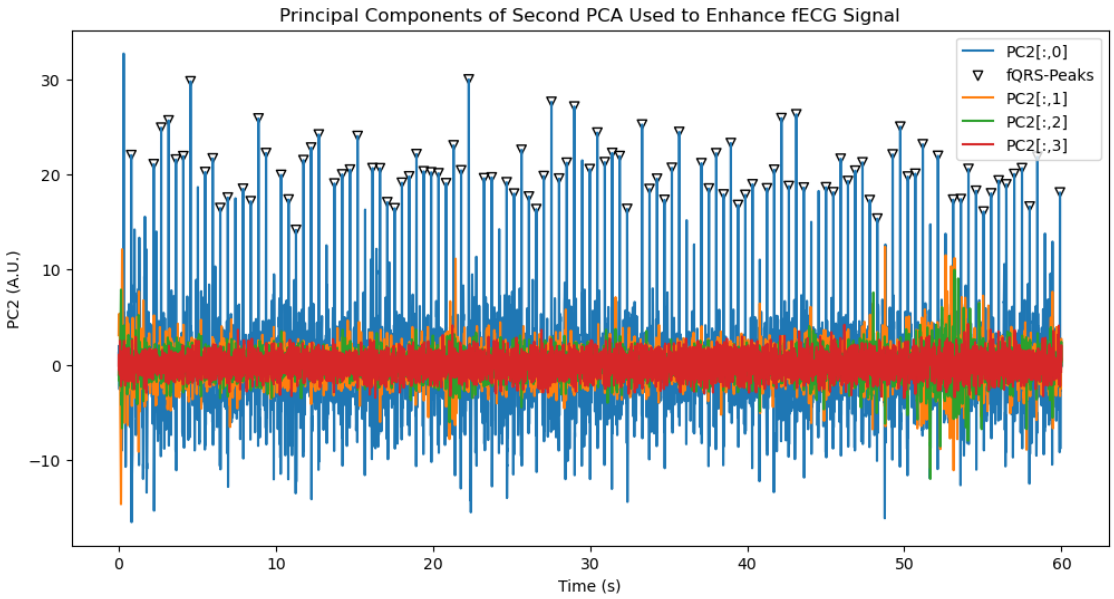


[6]:
```
# apply second PCA for enhancing fetal ECG component in residual signal
pca2 = PCA()
pc2 = pca2.fit_transform(x_residual)

# fetal ECG as the first principal component, note that this
# remains a guess and would need to be automated in the final solution
fetal_ecg = pc2[:, 0]
# detect fetal QRS peaks
fqrs_peaks = Detectors(fs).engzee_detector(fetal_ecg)

# plot
plt.figure(figsize=(12, 6))
for i in range(pc2.shape[1]):
    plt.plot(t, pc2[:, i], label=f'PC2[:,{i}]')
    if i == 0:
```

```
        plt.scatter(t[fqrs_peaks], pc2[fqrs_peaks, i], color='black',␣
 ↪marker='v', facecolors='none', label='fQRS-Peaks')
plt.xlabel('Time (s)')
plt.ylabel('PC2 (A.U.)')
plt.title('Principal Components of Second PCA Used to Enhance fECG Signal')
plt.legend()
plt.show()
```

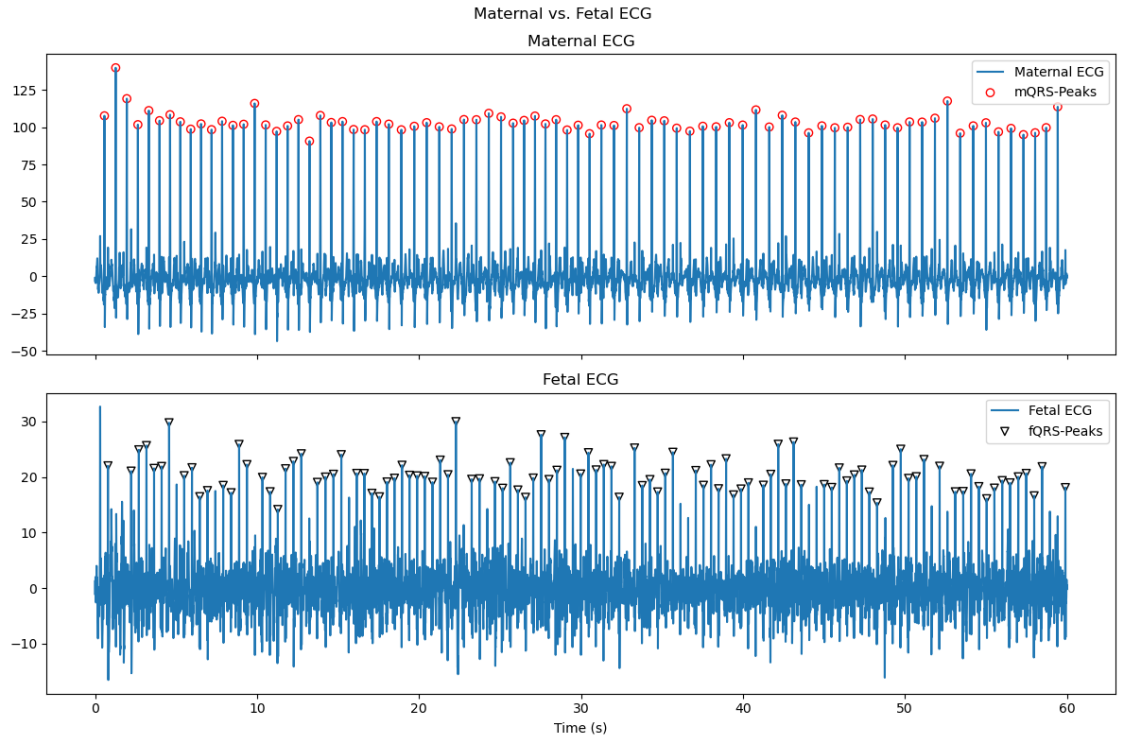


[7]:
```
# plot for summarizing all
fig, axs = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

axs[0].plot(t, maternal_ecg, label='Maternal ECG')
axs[0].scatter(t[mqrs_peaks], maternal_ecg[mqrs_peaks], color='red',␣
 ↪marker='o', facecolors='none', label='mQRS-Peaks')
axs[0].set_title('Maternal ECG')
axs[0].legend()

axs[1].plot(t, fetal_ecg, label='Fetal ECG')
axs[1].scatter(t[fqrs_peaks], fetal_ecg[fqrs_peaks], color='black', marker='v',␣
 ↪facecolors='none', label='fQRS-Peaks')
axs[1].set_title('Fetal ECG')
axs[1].set_xlabel('Time (s)')
axs[1].legend()

plt.suptitle('Maternal vs. Fetal ECG')
plt.tight_layout()
plt.show()
```



1.4 Exercise Questions

Please provide your answers directly below each question.

Question 1 Determine the maternal heart rate, both expressed in `Hz` and `beats/min`.

[8]:
```
# RR intervals and instantaneous MHR
mrr = np.diff(t[mqrs_peaks])
mhr_inst = 60 / mrr
mhr_times = t[mqrs_peaks][:-1]

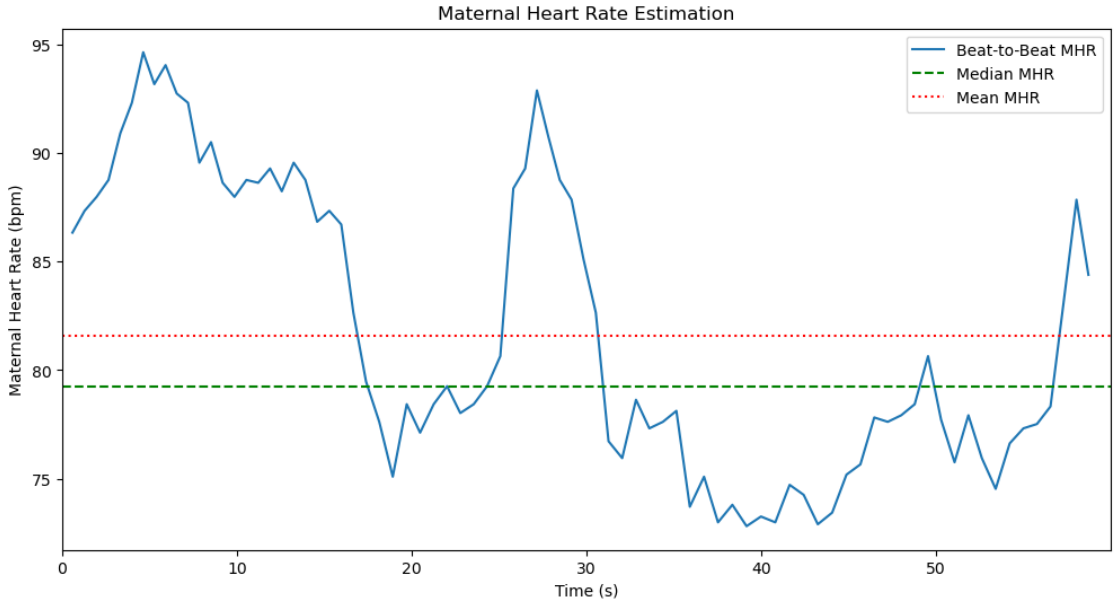
# MHR statistics
mhr_median = 60 / np.median(mrr)
mhr_mean = 60 / np.mean(mrr)

print(f'Maternal Heart rate : mean={mhr_mean:.1f} bpm; median={mhr_median:.1f}␣
↪bpm')
print(f'Maternal Heart rate : mean={mhr_mean/60:.1f} Hz; median={mhr_median/60:.
↪1f} Hz\n')
```

```
Maternal Heart rate : mean=81.6 bpm; median=79.3 bpm
Maternal Heart rate : mean=1.4 Hz; median=1.3 Hz
```

[9]:
```
# plot maternal heart rate
plt.figure(figsize=(12, 6))
plt.plot(mhr_times, mhr_inst, label="Beat-to-Beat MHR")
plt.axhline(y=mhr_median, color='g', linestyle='--', label="Median MHR")
plt.axhline(y=mhr_mean, color='r', linestyle=':', label="Mean MHR")

plt.xlabel("Time (s)")
plt.ylabel("Maternal Heart Rate (bpm)")
plt.xlim(t[0], t[-1])
plt.title("Maternal Heart Rate Estimation")
plt.legend()
plt.show()
```



Question 2 Determine the fetal heart rate, both expressed in `Hz` and `beats/min`.

[10]:
```
# RR intervals and instantaneous FHR
frr = np.diff(t[fqrs_peaks])
fhr_inst = 60 / frr
fhr_times = t[fqrs_peaks][:-1]

# FHR statistics
fhr_median = 60 / np.median(frr)
fhr_mean   = 60 / np.mean(frr)

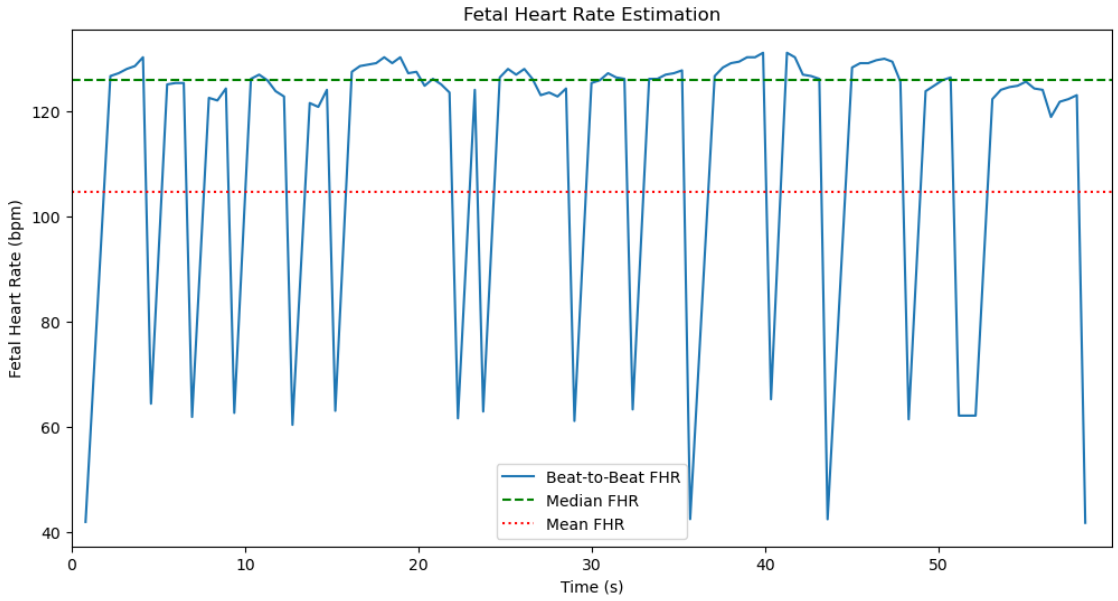
print(f'Fetal Heart rate : mean={fhr_mean:.1f} bpm; median={fhr_median:.1f}␣
 ↪bpm')
```

```
print(f'Fetal Heart rate : mean={fhr_mean/60:.1f} Hz; median={fhr_median/60:.
↪1f} Hz')
```

```
Fetal Heart rate : mean=104.6 bpm; median=125.8 bpm
Fetal Heart rate : mean=1.7 Hz; median=2.1 Hz
```

[11]:
```
# plot fetal heart rate
plt.figure(figsize=(12, 6))
plt.plot(fhr_times, fhr_inst, label="Beat-to-Beat FHR")
plt.axhline(y=fhr_median, color='g', linestyle='--', label="Median FHR")
plt.axhline(y=fhr_mean, color='r', linestyle=':', label="Mean FHR")

plt.xlabel("Time (s)")
plt.ylabel("Fetal Heart Rate (bpm)")
plt.xlim(t[0], t[-1])
plt.title("Fetal Heart Rate Estimation")
plt.legend()
plt.show()
```



Question 3 Determine the follwing three values:

- i) the average amplitude of the maternal QRS peaks (`mQRS`);
- ii) the average amplitude of the fetal QRS peaks (`fQRS`);
- iii) the ratio between the average amplitudes of i) `mQRS` and ii) `fQRS` peaks.

[12]:
```
#I)
mqrs_amp = maternal_ecg[mqrs_peaks]
mqrs_mean = np.mean(mqrs_amp)
```

```
mqrs_median = np.median(mqrs_amp)
print('Average amplitude of mQRS: mean={:.1f}; median={:.1f}'.format(mqrs_mean,␣
 ↪mqrs_median))
# II)
fqrs_amp = fetal_ecg[fqrs_peaks]
fqrs_mean = np.mean(fqrs_amp)
fqrs_median = np.median(fqrs_amp)
print('Average amplitude of fQRS: mean={:.1f}; median={:.1f}'.format(fqrs_mean,␣
 ↪fqrs_median))
# III)
print('Ratio mQRS/fQRS: {:2.2f}'.format(mqrs_median/fqrs_median))
```

```
Average amplitude of mQRS: mean=103.3; median=102.1
Average amplitude of fQRS: mean=20.5; median=20.2
Ratio mQRS/fQRS: 5.06
```

Question 4 How many of the principal components of the first PCA clearly show a maternal ECG signal? Which ones?

The first two components of the first PCA: **PC1[:, 0]** and **PC1[:, 1]** clearly show a maternal ECG signal.

Question 5 How many of the principal components of the second PCA clearly show a fetal ECG signal? Which ones?

Only one component of the second PCA clearly show a fetal ECG signal, being the first one: **PC2 [:, 0]**

Question 6 Not all of the fetal QRS peaks seem to be detected properly. Do you have an explanation why this happens and under which circumstances? Is it a problem of the fQRS detector, the mQRS cancellation or of another block of the algorithm?

We can infer that the mQRS cancellation is not perfect because nearly all the fQRS peaks that didn't get properly detected happened closely to the mRQS peaks.This indicates that when mQRS of high amplitude and fQRS of small amplitude have a similar shape, it is difficult to separate them. Moreover, missign detection can also be partly attributed to the QRS peak detector.

2 PCA for Source Separation of Ventilation and Cardiovascular Activity in Electrical Impedance Tomography (EIT)

2.1 Introduction

In this exercise we use PCA for processing image sequences of thoracic electrical impedance tomography (EIT) signals. EIT is a non-invasive, radiation-free imaging modality which uses small alternating currents to measure bioimpedance of the thorax [1]. These measurements are then converted into image sequences of thoracic impedance changes representing ventilation (i.e., air exchange in the lungs) and cardiovascular activity (e.g., heart movement or blood volume changes in heart and lungs).

In order to analyze these data it is important to properly separate ventilation and cardiovascular activity. Besides common techniques such as frequency filtering or ECG-triggered averaging, PCA can be used for separating these two sources of signals. The present example uses the method proposed by Deibele et al. [2] for which the block diagram is shown below:

2.2 References

[1] I. Frerichs et al., "Chest electrical impedance tomography examination, data analysis, terminology, clinical use and recommendations: consensus statement of the TRanslational EIT developmeNt stuDy group," Thorax, vol. 72, no. 1, pp. 83–93, Jan. 2017, doi: 10.1136/thoraxjnl-2016-208357.

[2] J. M. Deibele, H. Luepschen, and S. Leonhardt, "Dynamic separation of pulmonary and cardiac changes in electrical impedance tomography," Physiological Measurement, vol. 29, no. 6, pp. S1–S14, Jun. 2008, doi: 10.1088/0967-3334/29/6/S01.

2.3 PCA Analysis Results

[13]:
```
import numpy as np
from scipy.io import loadmat
from scipy.signal import filtfilt, butter
from scipy.linalg import eigh

import matplotlib.pyplot as plt

from ecgdetectors import Detectors
```

[14]:
```
# load data
data = loadmat('EIT_Data.mat')
t = data['tEit'].flatten()[data['IdxRange'].flatten().astype(bool)]
fs = 1/np.median(np.diff(t))
imgs_eit = data['Imgs']
b, a = butter(4, np.asarray([0.1, 12]), fs=fs, btype='bandpass')
imgs_eit = filtfilt(b, a, imgs_eit, axis=-1)
imgs_eit *= 1E3  # adapt scaling for plotting

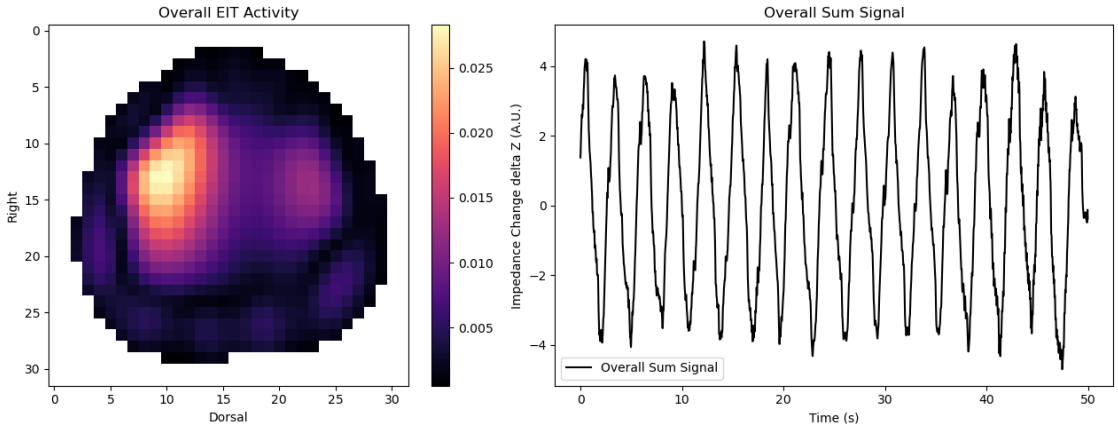
# ECG data to be used for bonus question
ecg = {'time': data['Ecg'][0][0][2], 'value': data['Ecg'][0][0][1], 'fs':␣
 ↪data['Ecg'][0][0][3]}
ecg_range = (ecg['time'] > t[0]) & (ecg['time'] < t[-1])
ecg['time'] = ecg['time'][ecg_range]
ecg['value'] = ecg['value'][ecg_range]

# force all timings to start at zero
t -= t[0]
ecg['time'] -= ecg['time'][0]
```

[15]:
```
# plot input data
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
```

```
im = axs[0].imshow(np.std(imgs_eit, axis=-1), cmap='magma', origin='upper')
axs[0].set_title('Overall EIT Activity')
axs[0].set_xlabel('Dorsal')
axs[0].set_ylabel('Right')
plt.colorbar(im, ax=axs[0], fraction=0.046, pad=0.04)

axs[1].plot(t, np.nansum(imgs_eit, axis=(0, 1)), color='black', label='Overall␣
 ↪Sum Signal')
axs[1].set_xlabel('Time (s)')
axs[1].set_ylabel('Impedance Change delta Z (A.U.)')
axs[1].set_title('Overall Sum Signal')
axs[1].legend()
plt.tight_layout()
plt.show()
```



[16]:
```
def compute_principal_components(X):
    # perform PCA and compute principal components (pc) and eigenvalues of X
    A = np.dot(X.transpose(), X)  # covariance matrix
    [eigenvalues, eigenvectors] = eigh(A)
    pc = np.dot(X, eigenvectors)
    return pc, eigenvalues

def estimate_cardiac_frequency(s, fs):
    sign_cardiac = np.sign(s)
    # find where zero crossings occured
    index = np.where(np.diff(sign_cardiac) == 2)[0]
    # interpolate to a sub-sample resolution
    pos = index + s[index + 1] / (s[index + 1] - s[index])
    # interpolate RR interval values
    rr = np.diff(pos) / fs
    return 1/np.median(rr)
```

```
def lms(A, B):
    if B.ndim == 1:
        B = np.asmatrix(B).transpose()
    tmp = np.linalg.solve(np.dot(B.transpose(), B), B.transpose())
    return np.dot(np.dot(B, tmp), A)

# separate ventilation and cardiovascular activity using PCA
# according to the algorithm by Deibele et al., PhysMeas, 2008
# https://dx.doi.org/10.1088/0967-3334/29/6/S01
imgs_tmp = np.reshape(imgs_eit, [-1, imgs_eit.shape[-1]])
are_valid_pixels = np.all(~np.isnan(imgs_tmp), 1)
X = imgs_tmp[are_valid_pixels, :].transpose()

# first approximation (see block diagram)
X = X - np.repeat(np.reshape(np.mean(X, 0), [1, -1]), X.shape[0], 0)
PC1, lambda1 = compute_principal_components(X)
Bv = PC1[:, -1]
Xv_ = lms(X, Bv)
Xc_ = X - Xv_

# second approximation (see block diagram)
Xc_ = Xc_ - np.repeat(np.reshape(np.mean(Xc_, 0), [1, -1]), Xc_.shape[0], 0)
b, a = butter(6, np.asarray([0.92, 4.6]), fs=fs, btype='bandpass')
Xc_bp = filtfilt(b, a, Xc_, axis=0)
PC2, lambda2 = compute_principal_components(Xc_bp)
Bc_ = PC2[:, -2:]
fc = estimate_cardiac_frequency(Bc_[:, 0], fs)

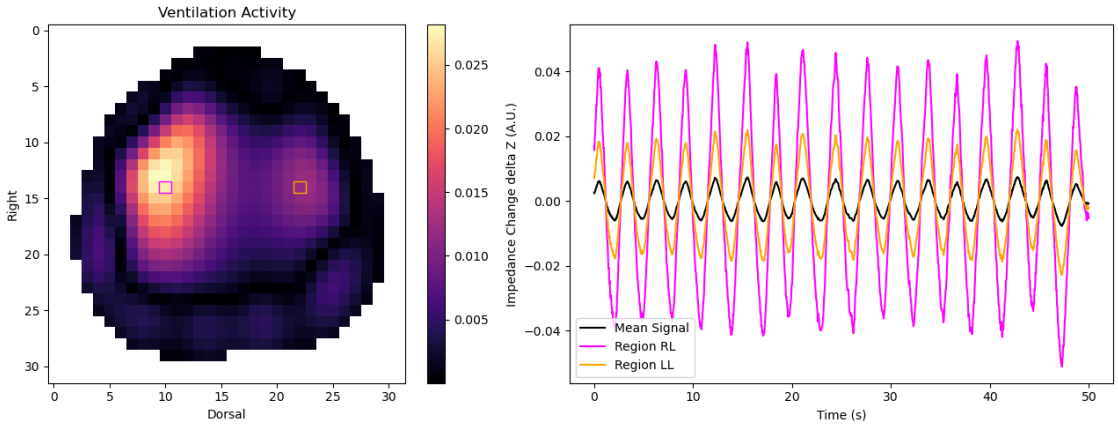
# create cardiac template functions
Bc = np.hstack((Bc_, np.roll(Bc_, int(fs/fc/3), 0),
                np.roll(Bc_, -int(fs/fc/3), 0)))
Xc1 = lms(Xc_, Bc)
Xc2 = lms(Xv_, Bc)
Xc = (Xc1 + Xc2).transpose()
Xv = (Xv_ - Xc2).transpose()

# cardiovascular activity
imgs_card = np.full(imgs_tmp.shape, np.nan)
imgs_card[are_valid_pixels, :] = Xc
imgs_card = imgs_card.reshape(imgs_eit.shape)
# ventilation activity
imgs_vent = np.full(imgs_tmp.shape, np.nan)
imgs_vent[are_valid_pixels, :] = Xv
imgs_vent = imgs_vent.reshape(imgs_eit.shape)
```

[17]:
```
# plot ventilation acticity
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
```

```
im = axs[0].imshow(np.std(imgs_vent, axis=-1), cmap='magma', origin='upper')
axs[0].set_title('Ventilation Activity')
axs[0].set_xlabel('Dorsal')
axs[0].set_ylabel('Right')
plt.colorbar(im, ax=axs[0], fraction=0.046, pad=0.04)

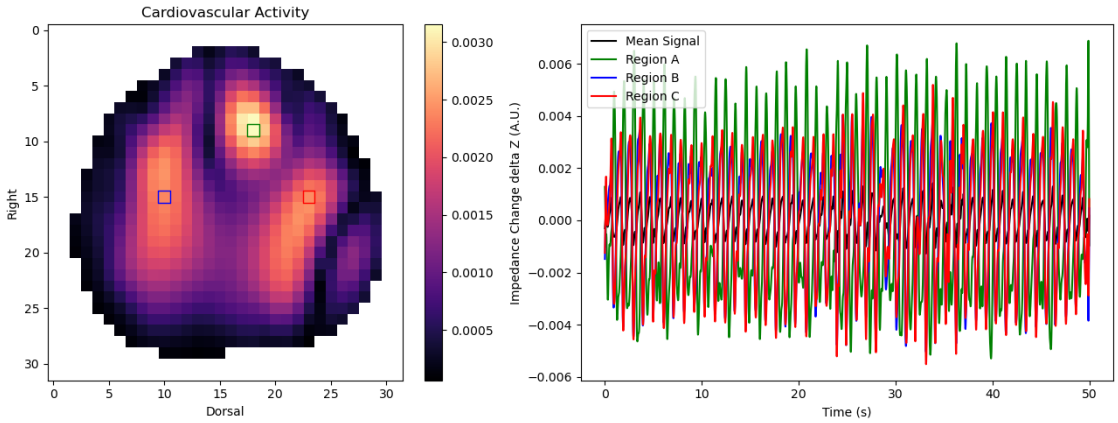
axs[1].plot(t, np.nanmean(imgs_vent, axis=(0, 1)), color='black', label='Mean␣
 ↪Signal')
regions = {'Region RL': ([10, 14], 'magenta'), 'Region LL': ([22, 14],␣
 ↪'orange')}
for reg, tmp in regions.items():
    axs[0].scatter(tmp[0][0], tmp[0][1], marker='s', s=100, edgecolor=tmp[1],␣
 ↪facecolor='none', label=reg)
    axs[1].plot(t, imgs_vent[tmp[0][1], tmp[0][0], :], color=tmp[1], label=reg)
axs[1].set_xlabel('Time (s)')
axs[1].set_ylabel('Impedance Change delta Z (A.U.)')
axs[1].legend()
plt.tight_layout()
plt.show()
```



[18]:
```
# plot cardiovascular acticity
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
im = axs[0].imshow(np.std(imgs_card, axis=-1), cmap='magma', origin='upper')
axs[0].set_title('Cardiovascular Activity')
axs[0].set_xlabel('Dorsal')
axs[0].set_ylabel('Right')
plt.colorbar(im, ax=axs[0], fraction=0.046, pad=0.04)

axs[1].plot(t, np.nanmean(imgs_card, axis=(0, 1)), color='black', label='Mean␣
 ↪Signal')
```

```
regions = {'Region A': ([18, 9], 'green'), 'Region B': ([10, 15], 'blue'),␣
 ↪'Region C': ([23, 15], 'red')}
for reg, tmp in regions.items():
    axs[0].scatter(tmp[0][0], tmp[0][1], marker='s', s=100, edgecolor=tmp[1],␣
 ↪facecolor='none', label=reg)
    axs[1].plot(t, imgs_card[tmp[0][1], tmp[0][0], :], color=tmp[1], label=reg)
axs[1].set_xlabel('Time (s)')
axs[1].set_ylabel('Impedance Change delta Z (A.U.)')
axs[1].legend()
plt.tight_layout()
plt.show()
```



2.4 Exercise Questions

Please provide your answers directly below each question.

Question 1 Determine the frequency of the ventilation activity (i.e., the respiratory rate), both expressed in `Hz` and `respirations/min`.

[19]:
```
vent_signal = np.nanmean(imgs_vent, axis=(0, 1))

from scipy.signal import filtfilt, butter
b, a = butter(4, 1.0/(fs/2))  # 4th order lowpass filter with cutoff at 1 Hz
vent_signal_filt = filtfilt(b, a, vent_signal)

# find negative-to-positive zero crossings (rising edge)
zc = (np.diff(vent_signal_filt)[1:] >= 0) & (np.diff(vent_signal_filt)[:-1] < 0)
t_zc = np.where(zc)[0] / fs

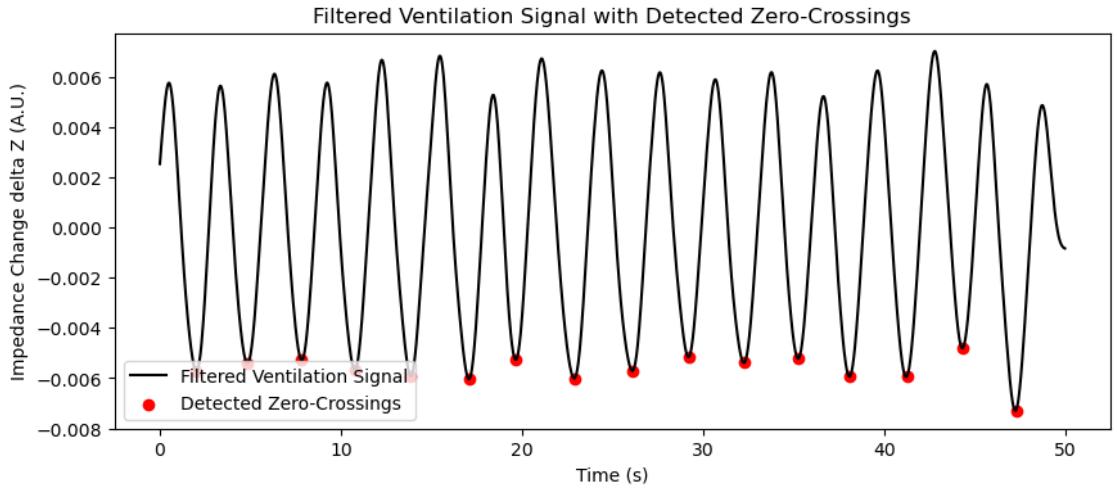
# compute RR intervals and respiratory rate
```

```
rr_intervals = np.diff(t_zc)
rr_mean = 60 / np.mean(rr_intervals)
rr_median = 60 / np.median(rr_intervals)

print(f'Respiratory rate (RR): mean={rr_mean:.1f} rpm; median={rr_median:.1f}␣
 ↪rpm')
print(f'Respiratory rate (RR): mean={rr_mean/60:.2f} Hz; median={rr_median/60:.
 ↪2f} Hz')
```

```
Respiratory rate (RR): mean=19.9 rpm; median=19.6 rpm
Respiratory rate (RR): mean=0.33 Hz; median=0.33 Hz
```

[20]:
```
# plotting the filtered ventilation signal with detected zero-crossings
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 4))
plt.plot(t, vent_signal_filt, label='Filtered Ventilation Signal',␣
 ↪color='black')
plt.scatter(t_zc, vent_signal_filt[np.where(zc)[0]+1], color='red',␣
 ↪label='Detected Zero-Crossings')
plt.title('Filtered Ventilation Signal with Detected Zero-Crossings')
plt.xlabel('Time (s)')
plt.ylabel('Impedance Change delta Z (A.U.)')
plt.legend()
plt.show()
```



Question 2 Determine the frequency of the cardiovascular activity, both expressed in `Hz` and `beats/min`.

[21]:
```
card_signal = np.nanmean(imgs_card, axis=(0, 1))
```

```
from scipy.signal import filtfilt, butter
b, a = butter(4, 1.0/(fs/2))  # 4th order lowpass filter with cutoff at 1 Hz
card_signal_filt = filtfilt(b, a, card_signal)

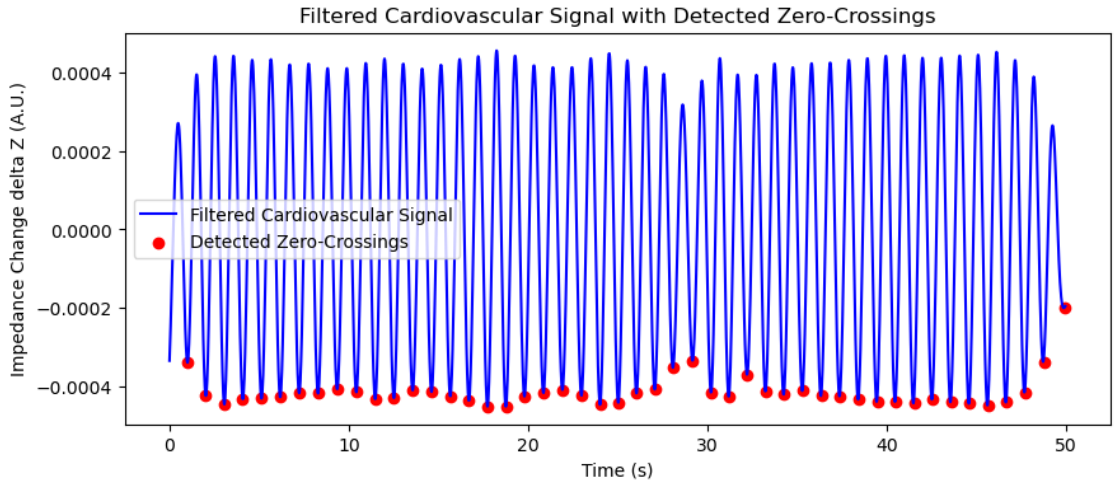
zc = (np.diff(card_signal_filt)[1:] >= 0) & (np.diff(card_signal_filt)[:-1] < 0)
t_zc = np.where(zc)[0] / fs

cr_intervals = np.diff(t_zc)
cr_mean = 60 / np.mean(cr_intervals)
cr_median = 60 / np.median(cr_intervals)

print(f'Cardiac rate (CR): mean={cr_mean:.1f} bpm; median={cr_median:.1f} bpm')
print(f'Cardiac rate (CR): mean={cr_mean/60:.2f} Hz; median={cr_median/60:.2f}␣
 ↪Hz')
```

```
Cardiac rate (CR): mean=57.6 bpm; median=58.3 bpm
Cardiac rate (CR): mean=0.96 Hz; median=0.97 Hz
```

[22]:
```
# plotting the filtered cardiovascular signal with detected zero-crossings
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 4))
plt.plot(t, card_signal_filt, label='Filtered Cardiovascular Signal',␣
 ↪color='blue')
plt.scatter(t_zc, card_signal_filt[np.where(zc)[0]+1], color='red',␣
 ↪label='Detected Zero-Crossings')
plt.title('Filtered Cardiovascular Signal with Detected Zero-Crossings')
plt.xlabel('Time (s)')
plt.ylabel('Impedance Change delta Z (A.U.)')
plt.legend()
plt.show()
```



Question 3 Determine the follwing three values:

- i) the maximal amplitude of ventilation activity;
- ii) the maximal amplitude of cardiovascular activity; and
- iii) the ratio between i) and ii), i.e., ventilation vs cardiovascular activity.

[23]:
```
vent_mean = np.nanmean(imgs_vent, axis=(0, 1))
card_mean = np.nanmean(imgs_card, axis=(0, 1))

vent_mean = vent_mean - np.mean(vent_mean)
card_mean = card_mean - np.mean(card_mean)

A_vent = np.max(np.abs(vent_mean))   # maximal ventilation amplitude
A_card = np.max(np.abs(card_mean))   # maximal cardiac amplitude
ratio  = A_card / A_vent             # cardiovascular vs ventilation

print(f'Maximal Ventilation Amplitude: {A_vent:.4f} ')
print(f'Maximal Cardiovascular Amplitude: {A_card:.4f} ')
print(f'Cardiovascular vs Ventilation Amplitude Ratio: {ratio:.2f}')
```

```
Maximal Ventilation Amplitude: 0.0077
Maximal Cardiovascular Amplitude: 0.0014
Cardiovascular vs Ventilation Amplitude Ratio: 0.19
```

Question 4 Determine the eigenvalues of the first three principal components resulting from the first PCA (see variable `PC1`).

[24]:
```
eig_pc1 = lambda1[-1]
eig_pc2 = lambda1[-2]
eig_pc3 = lambda1[-3]

print("Eigenvalues of first three PCs (first PCA):")
print("PC1:", eig_pc1)
print("PC2:", eig_pc2)
print("PC3:", eig_pc3)
```

```
Eigenvalues of first three PCs (first PCA):
PC1: 107.58688657123359
PC2: 2.111488879277311
PC3: 0.8031524385113595
```

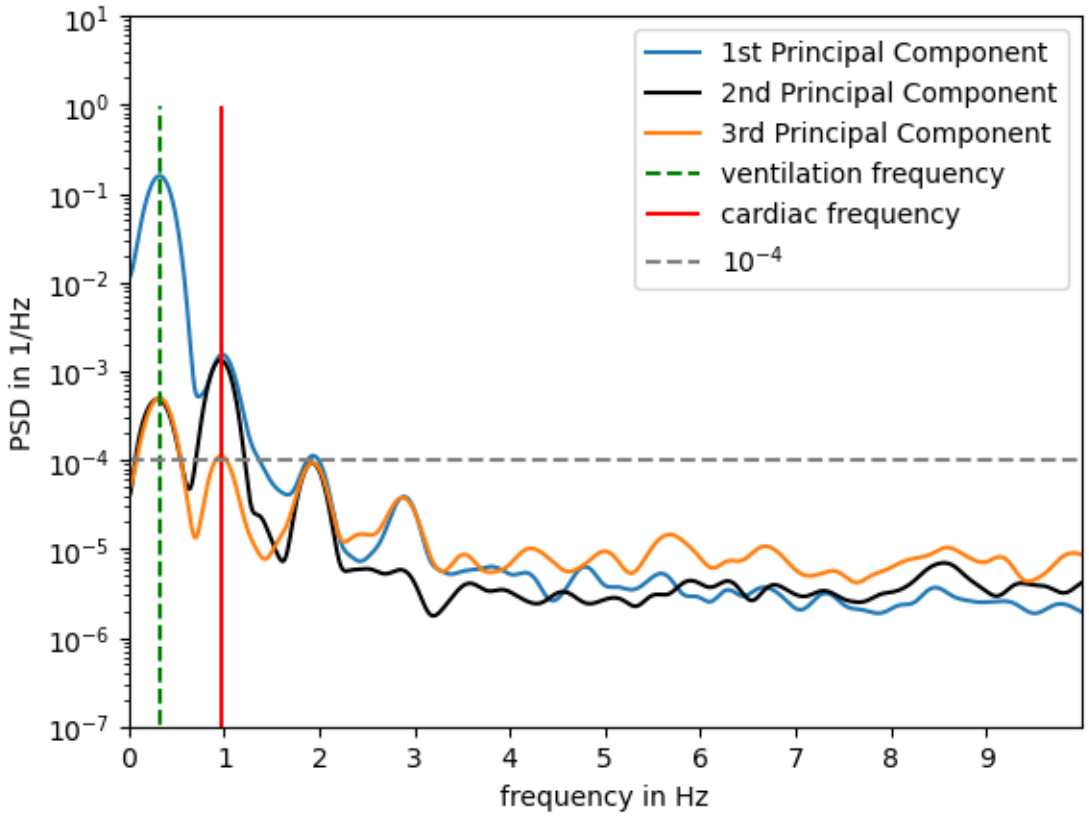
Question 5 Similar to Question 4, determine the two most dominant frequencies for the first three principal components. Note that this would lead to a total of 6 values, 2 frequencies for each of the 3 PCA components. However, for some components only one dominant frequency might be present. For all 3 PCA components, which one is rather related to ventilation or cardiovascular activity?

[25]:
```
from scipy.signal import welch
f, Pxxf = welch(PC1[:, -1], fs, return_onesided=True, nfft=2**14)
```

```
plt.semilogy(f, Pxxf, label='1st Principal Component')
f, Pxxf = welch(PC1[:, -2], fs, return_onesided=True, nfft=2**14)
plt.semilogy(f, Pxxf, label='2nd Principal Component',color="black")
f, Pxxf = welch(PC1[:, -3], fs, return_onesided=True, nfft=2**14)
plt.semilogy(f, Pxxf, label='3rd Principal Component')

plt.vlines(rr_median/60, plt.ylim()[0], plt.ylim()[-1],color='green',linestyle=␣
 ↪"dashed",label='ventilation frequency')
plt.vlines(cr_median/60, plt.ylim()[0], plt.ylim()[-1], color='red',␣
 ↪label='cardiac frequency')
plt.axhline(1e-4, color='gray', linestyle='--', label='10$^{-4}$')

plt.legend()
plt.xlabel('frequency in Hz')
plt.ylabel('PSD in 1/Hz')
plt.xlim([0, 10])
plt.xticks(np.arange(0, 10, 1))
plt.ylim([10**(-7), 10])
plt.show()
```



Dominant Frequencies and Associated Activities for First Three Principal Components

Principal Component	1st Dominant Frequency (Hz)	Associated Activity
PC1	[0.3, -]	Ventilation
PC2	[0.3, 1]	respiration and cardiac
PC3	[1, -]	cardiac

Question 6 The three example signals (Regions A to C: **green, blue, red**) of cardiovascular activity show the impedance change over time. Can you guess the underlying anatomical structure for each of the three example signals (Regions A to C: **green, blue, red**).

Region (Color)	Anatomical Structure
Green	Heart
Blue	Right lung
Red	Left lung

green is the heart because it has the highest cardiac activity (looking at the signal amplitude at the corresponding cardiac frequency). blue and red are lungs as they have a lower amplitude cardiac signal and they are in phase.

Question 7 - *Bonus Question* (not graded) Can you detect the QRS peaks (e.g., using `qrs = Detectors(fs).engzee_detector(ecg)`) on the ECG signal (see variable `ecg`) and plot them together with the cardiovascular EIT activity?

[26]:
```
# Detect QRS peaks on the ECG signal and plot them with the cardiovascular EIT␣
↪activity

# Detect QRS peaks
qrs_peaks = Detectors(int(ecg['fs'])).engzee_detector(ecg['value'].flatten())

# Compute mean cardiovascular EIT signal
card_signal = np.nanmean(imgs_card, axis=(0, 1))

# Plot ECG with QRS peaks and cardiovascular EIT activity
plt.figure(figsize=(14, 6))
ax1 = plt.gca()
ax1.plot(ecg['time'], ecg['value'], color='black', label='ECG')
ax1.scatter(ecg['time'][qrs_peaks], ecg['value'][qrs_peaks], color='red',␣
↪label='QRS Peaks', zorder=3)
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('ECG (A.U.)', color='black')
ax1.legend(loc='upper left')

ax2 = ax1.twinx()
ax2.plot(t, card_signal, color='blue', alpha=0.6, label='Cardiovascular EIT')
ax2.set_ylabel('Cardiovascular EIT (A.U.)', color='blue')
ax2.legend(loc='upper right')
```

```
plt.title('ECG with Detected QRS Peaks and Cardiovascular EIT Activity')
plt.tight_layout()
plt.show()
```

```
/var/folders/d1/9cx8z_ys2djcvm7tx40gw40r0000gn/T/ipykernel_17633/4143899861.py:4
: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is
deprecated, and will error in future. Ensure you extract a single element from
your array before performing this operation. (Deprecated NumPy 1.25.)
  qrs_peaks = Detectors(int(ecg['fs'])).engzee_detector(ecg['value'].flatten())
```

